

fbfrontend - Phase 10

UI polish and release verification

Complete source files and before/after changes

15 files: 11 updated and 4 added. Implemented locally after Phases 9.1-9.3.

Search now shows pending feedback immediately, cancels stale work and offers retry. Notifications have readable labels, and the browser title is FrndBook.

Mobile chat follows the visible viewport when available. Existing CSS fallback and safe-area padding remain. Real phone keyboard acceptance is still required.

Timestamp investigation found offset-free backend LocalDateTime values. Display preserves their clock fields; only explicitly zoned timestamps are converted. A backend/data decision is still needed for historical timezone conversion.

Verified: production build, 31 unit tests, new release browser checks, full 48-screen UI suite, and existing account/auth/chat/sidebar browser regressions. No lint errors; 12 warnings remain.

Mocked local services only. Backend and FrndBookWC2 unchanged. No cloud data changes, dependencies, migration or deployment.

Each changed file has full current source and a unified diff against the pre-Phase-10 snapshot. Green + lines are added; red - lines are removed. Gutter numbers are aids; long lines wrap with >.

The companion Plan and Testing Markdown provides detailed evidence, timezone findings, device/cloud acceptance checks, commands and rollback guidance.

File index

index.html	3
src/api/userApi.js	5
src/components/chat/ConversationList.jsx	8
src/components/chat/MessageList.jsx	11
src/components/users/FriendRequestCard.jsx	16
src/components/users/UserSearch.jsx	19
src/hooks/useChatViewport.js	27
src/pages/Messages.jsx	29
src/pages/Notifications.jsx	34
src/pages/Profile.jsx	39
src/pages/UserProfile.jsx	44
src/styles/chat.module.css	51
src/utils/displayText.js	60
tests/displayText.test.js	62
tests/release.browser.mjs	64

index.html

FULL CURRENT CONTENT

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <meta name="description" content="FrndBook – connect with friends and keep your conversations together." />
8     <title>FrndBook</title>
9   </head>
10  <body>
11    <div id="root"></div>
12    <script type="module" src="/src/main.jsx"></script>
13  </body>
14 </html>
```

index.html

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- before/index.html
2 +++ after/index.html
3 @@ -4,7 +4,8 @@
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7 -    <title>fbfrontend</title>
8 +    <meta name="description" content="FrndBook – connect with friends and keep your conversations together." />
9 +    <title>FrndBook</title>
10 </head>
11 <body>
12     <div id="root"></div>
```

```
1 import axiosClient from "../axiosClient";
2
3 // =====
4 // CURRENT USER
5 // =====
6
7 export const getCurrentUser = async () => {
8   const response = await axiosClient.get("/api/users/me");
9
10  return response.data;
11 };
12
13 // =====
14 // GET USER BY ID
15 // =====
16
17 export const getUserById = async (userId) => {
18   const response = await axiosClient.get(`/api/users/${userId}`);
19
20   return response.data;
21 };
22
23 // =====
24 // UPDATE PROFILE
25 // =====
26
27 export const updateProfile = async (name, bio) => {
28   const response = await axiosClient.patch("/api/users/me", {
29     name,
30     bio,
31   });
32
33   return response.data;
34 };
35
36 // =====
37 // UPDATE PROFILE IMAGE
38 // =====
39
40 export const updateProfileImage = async (file) => {
41   const formData = new FormData();
42
43   formData.append("file", file);
44
45   const response = await axiosClient.post(
46     "/api/users/me/profile-image",
47     formData,
48   );
49
50   return response.data;
51 };
52
53 // =====
54 // SEARCH USERS
55 // =====
56
57 export const searchUsers = async (name, page = 0, size = 10, signal) => {
58   const response = await axiosClient.get("/api/users/search", {
59     signal,
60     params: {
61       name,
62       page,
63       size,
64     },
65   });
66
67   return response.data;
68 };
69
70 // =====
```

```
71 // RECENT SEARCHES
72 // =====
73
74 export const getRecentSearches = async () => {
75   const response = await axiosClient.get("/api/users/recent-searches");
76
77   return response.data;
78 };
79
80 // =====
81 // ADD RECENT SEARCH
82 // =====
83
84 export const addRecentSearch = async (userId) => {
85   const response = await axiosClient.post(
86     `/api/users/recent-searches/${userId}`,
87   );
88
89   return response.data;
90 };
91
92 // =====
93 // CLEAR RECENT SEARCHES
94 // =====
95
96 export const clearRecentSearches = async () => {
97   const response = await axiosClient.delete("/api/users/recent-searches");
98
99   return response.data;
100 };
```

src/api/userApi.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/api/userApi.js
2  +++ after/src/api/userApi.js
3  @@ -54,8 +54,9 @@
4   // SEARCH USERS
5   // =====
6
7  -export const searchUsers = async (name, page = 0, size = 10) => {
8  +export const searchUsers = async (name, page = 0, size = 10, signal) => {
9     const response = await axiosClient.get("/api/users/search", {
10 +   signal,
11     params: {
12       name,
13       page,
```

```

1  import { formatTimestamp } from "../../utils/displayText.js";
2  import chatStyles from "../../styles/chat.module.css";
3  import { bindStyles } from "../../utils/bindStyles";
4  import UserAvatar from "../users/UserAvatar";
5
6  const css = bindStyles(chatStyles);
7
8  const ConversationList = ({
9    conversations,
10   selectedConversationId,
11   onSelect,
12   loading,
13 }) => {
14   return (
15     <aside className={css("conversation-list")}>
16       <div className={css("conversation-list-header")}>
17         <h2>Messages</h2>
18       </div>
19
20       {loading && conversations.length === 0 && (
21         <p className={css("chat-status")}>Loading conversations...</p>
22       )}
23
24       {!loading && conversations.length === 0 && (
25         <div className={css("conversation-empty")}>
26           <p>No conversations yet.</p>
27           <p>Open a friend and start a conversation.</p>
28         </div>
29       )}
30
31       <div className={css("conversation-items")}>
32         {conversations.map((conversation) => {
33           const otherUser = conversation.otherUser;
34
35           const isSelected =
36             String(conversation.id) === String(selectedConversationId);
37
38           return (
39             <button
40               key={conversation.id}
41               type="button"
42               className={css(`conversation-item ${
43                 isSelected ? "conversation-item-active" : ""
44               }`)}
45               aria-current={isSelected ? "true" : undefined}
46               onClick={() => onSelect(conversation)}
47             >
48               <UserAvatar
49                 name={otherUser?.name}
50                 image={otherUser?.profileImage}
51                 userId={otherUser?.id}
52                 size="medium"
53               />
54
55               <div className={css("conversation-item-content")}>
56                 <strong>{otherUser?.name || "Unknown User"}</strong>
57
58                 <span>
59                   {conversation.lastMessage?.content || "No messages yet"}
60                 </span>
61               </div>
62
63               <div className={css("conversation-item-time")}>
64                 {conversation.updatedAt
65                   ? formatTimestamp(conversation.updatedAt, true)
66                   : ""}
67               </div>
68             </button>
69           );
70         })}

```

src/components/chat/ConversationList.jsx

FULL CURRENT CONTENT

```
71         </div>
72       </aside>
73     );
74   };
75
76   export default ConversationList;
```

src/components/chat/ConversationList.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/components/chat/ConversationList.jsx
2  +++ after/src/components/chat/ConversationList.jsx
3  @@ -1,3 +1,4 @@
4  +import { formatTimestamp } from "../../utils/displayText.js";
5  import chatStyles from "../../styles/chat.module.css";
6  import { bindStyles } from "../../utils/bindStyles";
7  import UserAvatar from "../../users/UserAvatar";
8  @@ -61,10 +62,7 @@
9
10         <div className={css("conversation-item-time")}>
11             {conversation.updatedAt
12 -             ? new Date(conversation.updatedAt).toLocaleTimeString([], {
13 -                 hour: "2-digit",
14 -                 minute: "2-digit",
15 -             })
16 +             ? formatTimestamp(conversation.updatedAt, true)
17             : ""}
18         </div>
19     </button>
```

```

1  import { formatTimestamp } from "../../utils/displayText.js";
2  import chatStyles from "../../styles/chat.module.css";
3  import { bindStyles } from "../../utils/bindStyles";
4  import { useEffect, useLayoutEffect, useRef } from "react";
5
6  import UserAvatar from "../users/UserAvatar";
7
8  const css = bindStyles(chatStyles);
9
10 const MessageList = ({
11   messages,
12   currentUserId,
13   loading,
14   loadingOlder,
15   syncing = false,
16   historyError = "",
17   hasOlderMessages,
18   onLoadOlder,
19 }) => {
20   const messageListRef = useRef(null);
21
22   const preserveScrollRef = useRef(null);
23   const initialScrollPendingRef = useRef(false);
24   const previousMessageCountRef = useRef(0);
25   const isNearBottomRef = useRef(true);
26
27   const scrollToBottom = (behavior = "auto") => {
28     const element = messageListRef.current;
29
30     if (!element) {
31       return;
32     }
33
34     element.scrollTo({
35       top: element.scrollHeight,
36       behavior,
37     });
38   };
39
40   const updateNearBottom = () => {
41     const element = messageListRef.current;
42
43     if (!element) {
44       return;
45     }
46
47     const distanceFromBottom =
48       element.scrollHeight - element.scrollTop - element.clientHeight;
49
50     isNearBottomRef.current = distanceFromBottom <= 80;
51   };
52
53   const handleScroll = () => {
54     updateNearBottom();
55   };
56
57   /*
58   * When a conversation starts loading, prepare for the
59   * initial scroll-to-bottom after the first page arrives.
60   */
61   useEffect(() => {
62     if (loading) {
63       initialScrollPendingRef.current = true;
64       preserveScrollRef.current = null;
65     }
66   }, [loading]);
67
68   /*
69   * Handle all scroll positioning after the DOM has updated.
70   */

```

```

71  * 1. Older messages:
72  *   Preserve the user's visual position.
73  *
74  * 2. Initial load:
75  *   Start at the newest message.
76  *
77  * 3. New realtime messages:
78  *   Scroll only when the user is already near the bottom.
79  */
80  useEffect(() => {
81    if (loading) {
82      return;
83    }
84
85    const element = messageListRef.current;
86
87    if (!element) {
88      return;
89    }
90
91    /*
92     * Older messages were prepended.
93     *
94     * Keep the same message in approximately the same
95     * visual position instead of jumping to the bottom.
96     */
97    if (loadingOlder && preserveScrollRef.current) return;
98    if (!loadingOlder && preserveScrollRef.current) {
99      const previousScroll = preserveScrollRef.current;
100
101      const heightDifference =
102        element.scrollHeight - previousScroll.scrollHeight;
103
104      element.scrollTop = previousScroll.scrollTop + heightDifference;
105
106      preserveScrollRef.current = null;
107      previousMessageCountRef.current = messages.length;
108
109      updateNearBottom();
110
111      return;
112    }
113
114    /*
115     * First successful conversation load.
116     */
117    if (initialScrollPendingRef.current) {
118      scrollToBottom("auto");
119
120      initialScrollPendingRef.current = false;
121      previousMessageCountRef.current = messages.length;
122
123      updateNearBottom();
124
125      return;
126    }
127
128    /*
129     * New realtime message.
130     *
131     * Do not disturb someone who is reading older messages.
132     */
133    if (messages.length !== previousMessageCountRef.current) {
134      if (isNearBottomRef.current) {
135        scrollToBottom("smooth");
136      }
137
138      previousMessageCountRef.current = messages.length;
139
140      updateNearBottom();

```

```

141     }
142   }, [messages.length, loading, loadingOlder]);
143
144   const handleLoadOlder = async () => {
145     const element = messageListRef.current;
146
147     if (element) {
148       preserveScrollRef.current = {
149         scrollTop: element.scrollTop,
150         scrollHeight: element.scrollHeight,
151       };
152     }
153
154     try {
155       await onLoadOlder();
156     } catch {
157       /*
158        * Parent handles the actual error.
159        *
160        * Keeping this catch prevents the button handler
161        * from producing an unhandled promise rejection.
162        */
163     }
164   };
165
166   if (loading) {
167     return (
168       <div className={css("message-list-container")}>
169         <div className={css("message-list message-list-loading")}>
170           Loading messages...
171         </div>
172       </div>
173     );
174   }
175
176   return (
177     <div className={css("message-list-container")}>
178       {hasOlderMessages && (
179         <div className={css("older-messages-bar")}>
180           <button
181             type="button"
182             className={css("chat-secondary-button")}
183             onClick={handleLoadOlder}
184             disabled={loadingOlder || syncing}
185           >
186             {loadingOlder
187               ? "Loading older messages..."
188               : "↑ See older messages"}
189           </button>
190         </div>
191       )}
192
193       <div
194         ref={messageListRef}
195         className={css("message-list")}
196         onScroll={handleScroll}
197       >
198         {messages.length === 0 && (
199           <div className={css("message-empty")}>
200             <p>{historyError ? "Message history is unavailable. Please retry above." : "No messages yet."}</p>
201             {!historyError && <p>Say hello 🐼</p>}}
202           </div>
203         )}
204
205         {messages.map((message) => {
206           const isOwnMessage =
207             String(message.sender?.id) === String(currentUserId);
208
209           return (
210             <div

```

```

211         key={message.id}
212         className={css(`message-row ${
213           isOwnMessage ? "message-row-own" : "message-row-other"
214         })}
215       >
216         {!isOwnMessage && (
217           <UserAvatar
218             name={message.sender?.name}
219             image={message.sender?.profileImage}
220             userId={message.sender?.id}
221             size="small"
222           />
223         )}
224
225         <div
226           className={css(`message-bubble ${
227             isOwnMessage ? "message-bubble-own" : "message-bubble-other"
228           })}
229         >
230           <p>{message.content}</p>
231
232           <span>
233             {message.createdAt
234               ? formatTimestamp(message.createdAt, true)
235               : ""}
236           </span>
237         </div>
238       </div>
239     );
240   }}}
241 </div>
242 </div>
243 );
244 };
245
246 export default MessageList;

```

src/components/chat/MessageList.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- before/src/components/chat/MessageList.jsx
2 +++ after/src/components/chat/MessageList.jsx
3 @@ -1,3 +1,4 @@
4 +import { formatTimestamp } from "../../utils/displayText.js";
5 import chatStyles from "../../styles/chat.module.css";
6 import { bindStyles } from "../../utils/bindStyles";
7 import { useEffect, useLayoutEffect, useRef } from "react";
8 @@ -230,10 +231,7 @@
9
10         <span>
11             {message.createdAt
12 -                 ? new Date(message.createdAt).toLocaleTimeString([], {
13 -                     hour: "2-digit",
14 -                     minute: "2-digit",
15 -                 })
16 +                 ? formatTimestamp(message.createdAt, true)
17                 : ""}
18         </span>
19     </div>
```

```

1  import { formatTimestamp } from "../../utils/displayText.js";
2  import friendsStyles from "../../styles/friends.module.css";
3  import { bindStyles } from "../../utils/bindStyles";
4  import { Link } from "react-router-dom";
5
6  import UserAvatar from "../UserAvatar";
7
8  const css = bindStyles(friendsStyles);
9
10 const FriendRequestCard = ({
11   request,
12   type,
13   onAccept,
14   onReject,
15   actionLoading = false,
16 }) => {
17   if (!request) {
18     return null;
19   }
20
21   const isReceived = type === "received";
22
23   const person = isReceived ? request.sender : request.receiver;
24
25   if (!person) {
26     return null;
27   }
28
29   return (
30     <div className={css("friend-request-card")}>
31       <Link to={`/users/${person.id}`} className={css("friend-request-main")}>
32         <UserAvatar
33           name={person.name}
34           image={person.profileImage}
35           userId={person.id}
36           size="medium"
37         />
38
39         <div className={css("friend-request-info")}>
40           <h3>{person.name || "Unknown User"}</h3>
41
42           <p>{person.bio || "No bio available"}</p>
43
44           <span>
45             {request.createdAt
46               ? formatTimestamp(request.createdAt)
47               : "-"}
48           </span>
49         </div>
50       </Link>
51
52       {isReceived ? (
53         <div className={css("friend-request-actions")}>
54           <button
55             type="button"
56             className={css("friend-action-button primary")}
57             onClick={() => onAccept(request.id)}
58             disabled={actionLoading}
59           >
60             {actionLoading ? "Accepting..." : "Accept"}
61           </button>
62
63           <button
64             type="button"
65             className={css("friend-action-button secondary")}
66             onClick={() => onReject(request.id)}
67             disabled={actionLoading}
68           >
69             Reject
70           </button>

```

src/components/users/FriendRequestCard.jsx

FULL CURRENT CONTENT

```
71         </div>
72       ) : (
73         <span className={css("friend-request-sent-label")}>Request Sent</span>
74       )}
75     </div>
76   );
77 };
78
79 export default FriendRequestCard;
```

src/components/users/FriendRequestCard.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/components/users/FriendRequestCard.jsx
2  +++ after/src/components/users/FriendRequestCard.jsx
3  @@ -1,3 +1,4 @@
4  +import { formatTimestamp } from "../../utils/displayText.js";
5  import friendsStyles from "../../styles/friends.module.css";
6  import { bindStyles } from "../../utils/bindStyles";
7  import { Link } from "react-router-dom";
8  @@ -42,7 +43,7 @@
9
10         <span>
11             {request.createdAt
12 -             ? new Date(request.createdAt).toLocaleString()
13 +             ? formatTimestamp(request.createdAt)
14             : "-"}
15         </span>
16     </div>
```

```

1  import { apiErrorMessage } from "../../utils/apiError.js";
2  import usersStyles from "../../styles/users.module.css";
3  import { bindStyles } from "../../utils/bindStyles";
4  import StatusMessage from "../../ui/StatusMessage";
5  import { useEffect, useState } from "react";
6
7  import {
8    addRecentSearch,
9    clearRecentSearches,
10   getRecentSearches,
11   searchUsers,
12 } from "../../api/userApi";
13
14 import { useAuth } from "../../auth/AuthContext";
15
16 import UserCard from "../UserCard";
17
18 const css = bindStyles(usersStyles);
19
20 const UserSearch = () => {
21   const { user: currentUser } = useAuth();
22
23   const [searchTerm, setSearchTerm] = useState("");
24
25   const [results, setResults] = useState([]);
26
27   const [recentSearches, setRecentSearches] = useState([]);
28
29   const [page, setPage] = useState(0);
30   const [requestedPage, setRequestedPage] = useState(0);
31   const [requestVersion, setRequestVersion] = useState(0);
32   const [clearing, setClearing] = useState(false);
33   const [recentError, setRecentError] = useState("");
34
35   const [totalPages, setTotalPages] = useState(0);
36
37   const [loading, setLoading] = useState(false);
38
39   const [recentLoading, setRecentLoading] = useState(true);
40
41   const [error, setError] = useState("");
42
43   // =====
44   // LOAD RECENT SEARCHES
45   // =====
46
47   useEffect(() => {
48     let active = true;
49     const loadRecentSearches = async () => {
50       try {
51         setRecentLoading(true);
52
53         const data = await getRecentSearches();
54
55         if (active) setRecentSearches(data || []);
56       } catch (error) {
57         if (active) setRecentError(apiErrorMessage(error, "Unable to load recent searches"));
58       } finally {
59         if (active) setRecentLoading(false);
60       }
61     };
62
63     loadRecentSearches();
64     return () => { active = false; };
65   }, []);
66
67   // One effect owns debouncing, paging and cancellation for each search.
68   useEffect(() => {
69     const query = searchTerm.trim();
70     if (!query) return;

```

```

71     const abort = new AbortController();
72     const timer = setTimeout(async () => {
73       try {
74         const data = await searchUsers(query, requestedPage, 10, abort.signal);
75         if (abort.signal.aborted) return;
76         setResults((data?.content || []).filter(item => String(item.id) !== String(currentUser?.id)));
77         setPage(data?.number ?? requestedPage);
78         setTotalPages(data?.totalPages ?? 0);
79       } catch (error) {
80         if (!abort.signal.aborted) {
81           setError(apiErrorMessage(error, "Unable to search users"));
82           setResults([]);
83         }
84       } finally { if (!abort.signal.aborted) setLoading(false); }
85     }, requestedPage === 0 ? 400 : 0);
86     return () => { clearTimeout(timer); abort.abort(); };
87   }, [searchTerm, requestedPage, requestVersion, currentUser?.id]);
88
89   const changeSearch = value => {
90     setSearchTerm(value); setRequestedPage(0); setPage(0); setTotalPages(0);
91     setResults([]); setError(""); setLoading(Boolean(value.trim()));
92   };
93   const loadPage = nextPage => {
94     if (loading || nextPage === page) return;
95     setError(""); setLoading(true); setRequestedPage(nextPage); setRequestVersion(version => version + 1);
96   };
97
98   // =====
99   // USER CLICK
100  // =====
101
102  const handleUserClick = async (selectedUser) => {
103    try {
104      await addRecentSearch(selectedUser.id);
105
106      /*
107       * Refresh recent searches so the UI
108       * immediately reflects the backend state.
109       */
110
111      const updatedRecentSearches = await getRecentSearches();
112
113      setRecentSearches(updatedRecentSearches || []);
114    } catch (error) {
115      console.error("Failed to save recent search:", error);
116    }
117  };
118
119  // =====
120  // CLEAR RECENT SEARCHES
121  // =====
122
123  const handleClearRecentSearches = async () => {
124    if (clearing) return;
125    setClearing(true); setRecentError("");
126    try {
127      await clearRecentSearches();
128
129      setRecentSearches([]);
130    } catch (error) {
131      setRecentError(apiErrorMessage(error, "Unable to clear recent searches"));
132    } finally { setClearing(false); }
133  };
134
135  return (
136    <section className={css("user-search")}>
137      <div className={css("section-header")}>
138        <h2>Find People</h2>
139      </div>
140  
```

```

141     <input
142       type="text"
143       className={css("search-input")}
144       aria-label="Search people by name"
145       placeholder="Search users by name..."
146       value={searchTerm}
147       onChange={(event) => changeSearch(event.target.value)}
148     />
149
150     {error && <StatusMessage tone="error" className={css("error")}>{error}
151       <button type="button" onClick={() => { setError(""); setLoading(true); setRequestVersion(version => versi
152     > on + 1); }}>Retry search</button>
153     </StatusMessage>}
154
155     {loading && <p role="status" className={css("search-status")}>Searching...</p>}
156
157     {!loading && searchTerm.trim() && results.length === 0 && !error && (
158       <p className={css("search-status")}>No users found.</p>
159     )}
160
161     <div aria-busy={loading} className={css("user-results")}>
162       {results.map((searchedUser) => (
163         <UserCard
164           key={searchedUser.id}
165           user={searchedUser}
166           onClick={handleUserClick}
167         />
168       ))}
169     </div>
170
171     {totalPages > 1 && searchTerm.trim() && (
172       <div className={css("pagination")}>
173         <button
174           disabled={loading || page === 0}
175           onClick={() => loadPage(page - 1)}
176         >
177           Previous
178         </button>
179
180         <span>
181           Page {page + 1} of {totalPages}
182         </span>
183
184         <button
185           disabled={loading || page >= totalPages - 1}
186           onClick={() => loadPage(page + 1)}
187         >
188           Next
189         </button>
190       </div>
191     )}
192
193     {!searchTerm.trim() && (
194       <section className={css("recent-searches")}>
195         <div className={css("section-header")}>
196           <h3>Recent Searches</h3>
197
198           {recentSearches.length > 0 && (
199             <button
200               className={css("text-button")}
201               onClick={handleClearRecentSearches}
202               disabled={clearing}
203             >
204               {clearing ? "Clearing..." : "Clear"}
205             </button>
206           )}
207         </div>
208
209         {recentError && <StatusMessage tone="error">{recentError}</StatusMessage>}

```

```
210      {recentLoading && (  
211        <p className={css("search-status")}>Loading recent searches...</p>  
212      )}  
213  
214      {!recentLoading && !recentError && recentSearches.length === 0 && (  
215        <p className={css("search-status")}>No recent searches.</p>  
216      )}  
217  
218      <div className={css("user-results")}>  
219        {recentSearches.map((recentUser) => (  
220          <UserCard  
221            key={recentUser.id}  
222            user={recentUser}  
223            onClick={handleUserClick}  
224          />  
225        ))}  
226      </div>  
227    </section>  
228  )}  
229 </section>  
230 );  
231 };  
232  
233 export default UserSearch;
```

src/components/users/UserSearch.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/components/users/UserSearch.jsx
2  +++ after/src/components/users/UserSearch.jsx
3  @@ -27,6 +27,10 @@
4      const [recentSearches, setRecentSearches] = useState([]);
5
6      const [page, setPage] = useState(0);
7  +   const [requestedPage, setRequestedPage] = useState(0);
8  +   const [requestVersion, setRequestVersion] = useState(0);
9  +   const [clearing, setClearing] = useState(false);
10  +   const [recentError, setRecentError] = useState("");
11
12     const [totalPages, setTotalPages] = useState(0);
13
14  @@ -41,116 +45,54 @@
15     // =====
16
17     useEffect(() => {
18  +   let active = true;
19     const loadRecentSearches = async () => {
20       try {
21         setRecentLoading(true);
22
23         const data = await getRecentSearches();
24
25  -       setRecentSearches(data || []);
26  +       if (active) setRecentSearches(data || []);
27       } catch (error) {
28  -       console.error("Failed to load recent searches:", error);
29  +       if (active) setRecentError(apiErrorMessage(error, "Unable to load recent searches"));
30       } finally {
31  -       setRecentLoading(false);
32  +       if (active) setRecentLoading(false);
33     }
34   };
35
36   loadRecentSearches();
37  +   return () => { active = false; };
38   }, []);
39
40  - // =====
41  - // SEARCH
42  - // =====
43  -
44  + // One effect owns debouncing, paging and cancellation for each search.
45   useEffect(() => {
46  -   const trimmedSearch = searchTerm.trim();
47  -
48  -   if (!trimmedSearch) {
49  -     setResults([]);
50  -
51  -     setPage(0);
52  -
53  -     setTotalPages(0);
54  -
55  -     setError("");
56  -
57  -     return;
58  -   }
59  -
60  +   const query = searchTerm.trim();
61  +   if (!query) return;
62  +   const abort = new AbortController();
63     const timer = setTimeout(async () => {
64       try {
65  -       setLoading(true);
66  -
67  -       setError("");
68  -
69  -       const data = await searchUsers(trimmedSearch, 0, 10);
70  -
```

```

71 -     const users = data?.content || [];
72 -
73 -     /*
74 -     * Don't show the currently logged-in
75 -     * user in user search.
76 -     */
77 -
78 -     const filteredUsers = users.filter(
79 -       (searchedUser) => searchedUser.id !== currentUser?.id,
80 -     );
81 -
82 -     setResults(filteredUsers);
83 -
84 -     setPage(data?.number ?? 0);
85 -
86 +     const data = await searchUsers(query, requestedPage, 10, abort.signal);
87 +     if (abort.signal.aborted) return;
88 +     setResults((data?.content || []).filter(item => String(item.id) !== String(currentUser?.id)));
89 +     setPage(data?.number ?? requestedPage);
90 +     setTotalPages(data?.totalPages ?? 0);
91   } catch (error) {
92 -     console.error("User search failed:", error);
93 -
94 -     setError(apiErrorMessage(error, "Unable to search users"));
95 -
96 -     setResults([]);
97 -   } finally {
98 -     setLoading(false);
99 -   }
100 - }, 400);
101 -
102 -   return () => {
103 -     clearTimeout(timer);
104 -   };
105 - }, [searchTerm, currentUser?.id]);
106 -
107 - // =====
108 - // LOAD PAGE
109 - // =====
110 -
111 - const loadPage = async (nextPage) => {
112 -   if (!searchTerm.trim()) {
113 -     return;
114 -   }
115 -
116 -   try {
117 -     setLoading(true);
118 -
119 -     setError("");
120 -
121 -     const data = await searchUsers(searchTerm.trim(), nextPage, 10);
122 -
123 -     const users = data?.content || [];
124 -
125 -     const filteredUsers = users.filter(
126 -       (searchedUser) => searchedUser.id !== currentUser?.id,
127 -     );
128 -
129 -     setResults(filteredUsers);
130 -
131 -     setPage(data?.number ?? nextPage);
132 -
133 -     setTotalPages(data?.totalPages ?? 0);
134 -   } catch (error) {
135 -     console.error("Failed to load search page:", error);
136 -
137 -     setError(apiErrorMessage(error, "Unable to load results"));
138 -   } finally {
139 -     setLoading(false);
140 -   }

```

```

141 +     if (!abort.signal.aborted) {
142 +       setError(apiErrorMessage(error, "Unable to search users"));
143 +       setResults([]);
144 +     }
145 +   } finally { if (!abort.signal.aborted) setLoading(false); }
146 + }, requestedPage === 0 ? 400 : 0);
147 + return () => { clearTimeout(timer); abort.abort(); };
148 + }, [searchTerm, requestedPage, requestVersion, currentUser?.id]);
149 +
150 + const changeSearch = value => {
151 +   setSearchTerm(value); setRequestedPage(0); setPage(0); setTotalPages(0);
152 +   setResults([]); setError(""); setLoading(Boolean(value.trim()));
153 + };
154 + const loadPage = nextPage => {
155 +   if (loading || nextPage === page) return;
156 +   setError(""); setLoading(true); setRequestedPage(nextPage); setRequestVersion(version => version + 1);
157 + };
158
159 // =====
160 @@ -179,13 +121,15 @@
161 // =====
162
163 const handleClearRecentSearches = async () => {
164 +   if (clearing) return;
165 +   setClearing(true); setRecentError("");
166   try {
167     await clearRecentSearches();
168
169     setRecentSearches([]);
170   } catch (error) {
171 -     console.error("Failed to clear recent searches:", error);
172 -   }
173 +   setRecentError(apiErrorMessage(error, "Unable to clear recent searches"));
174 +   } finally { setClearing(false); }
175 };
176
177 return (
178 @@ -200,18 +144,20 @@
179   aria-label="Search people by name"
180   placeholder="Search users by name..."
181   value={searchTerm}
182 -   onChange={(event) => setSearchTerm(event.target.value)}
183 +   onChange={(event) => changeSearch(event.target.value)}
184   />
185
186 -   {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
187 -
188 -   {loading && <p className={css("search-status")}>Searching...</p>}
189 +   {error && <StatusMessage tone="error" className={css("error")}>{error}
190 +     <button type="button" onClick={() => { setError(""); setLoading(true); setRequestVersion(version => vers
191 +       ion + 1); }}>Retry search</button>
192 +   </StatusMessage>}
193 +   {loading && <p role="status" className={css("search-status")}>Searching...</p>}
194
195   {!loading && searchTerm.trim() && results.length === 0 && !error && (
196     <p className={css("search-status")}>No users found.</p>
197   )}
198
199 -   <div className={css("user-results")}>
200 +   <div aria-busy={loading} className={css("user-results")}>
201     {results.map((searchedUser) => (
202       <UserCard
203         key={searchedUser.id}
204 @@ -252,17 +198,20 @@
205       <button
206         className={css("text-button")}
207         onClick={handleClearRecentSearches}
208 +       disabled={clearing}
209     >

```

src/components/users/UserSearch.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
210 -         Clear
211 +         {clearing ? "Clearing..." : "Clear"}
212         </button>
213     )}
214 </div>
215
216 +     {recentError && <StatusMessage tone="error">{recentError}</StatusMessage>}
217 +
218     {recentLoading && (
219         <p className={css("search-status")}>Loading recent searches...</p>
220     )}
221
222 -     {!recentLoading && recentSearches.length === 0 && (
223 +     {!recentLoading && !recentError && recentSearches.length === 0 && (
224         <p className={css("search-status")}>No recent searches.</p>
225     )}
226
```

```
1 import { useEffect, useState } from "react";
2
3 /** Fit the mobile chat to the visible area when an on-screen keyboard overlays the layout. */
4 export function useChatViewport() {
5   const [element, setElement] = useState(null);
6   useEffect(() => {
7     const viewport = window.visualViewport;
8     if (!element || !viewport) return;
9     let frame;
10    const measure = () => {
11      cancelAnimationFrame(frame);
12      frame = requestAnimationFrame(() => {
13        if (window.innerWidth > 800 || viewport.scale !== 1) {
14          element.style.removeProperty("--chat-available-height");
15          return;
16        }
17        const available = Math.max(0, viewport.height + viewport.offsetTop - element.getBoundingClientRect().top)
18      > ;
19      element.style.setProperty("--chat-available-height", `${available}px`);
20    });
21    measure();
22    viewport.addEventListener("resize", measure);
23    viewport.addEventListener("scroll", measure);
24    window.addEventListener("resize", measure);
25    window.addEventListener("scroll", measure);
26    return () => {
27      cancelAnimationFrame(frame);
28      viewport.removeEventListener("resize", measure);
29      viewport.removeEventListener("scroll", measure);
30      window.removeEventListener("resize", measure);
31      window.removeEventListener("scroll", measure);
32      element.style.removeProperty("--chat-available-height");
33    };
34  }, [element]);
35  return setElement;
36 }
```

```
1 --- /dev/null
2 +++ after/src/hooks/useChatViewport.js
3 @@ -0,0 +1,36 @@
4 +import { useEffect, useState } from "react";
5 +
6 +/** Fit the mobile chat to the visible area when an on-screen keyboard overlays the layout. */
7 +export function useChatViewport() {
8 +  const [element, setElement] = useState(null);
9 +  useEffect(() => {
10 +    const viewport = window.visualViewport;
11 +    if (!element || !viewport) return;
12 +    let frame;
13 +    const measure = () => {
14 +      cancelAnimationFrame(frame);
15 +      frame = requestAnimationFrame(() => {
16 +        if (window.innerWidth > 800 || viewport.scale !== 1) {
17 +          element.style.removeProperty("--chat-available-height");
18 +          return;
19 +        }
20 +        const available = Math.max(0, viewport.height + viewport.offsetTop - element.getBoundingClientRect().top
21 + > );
22 +        element.style.setProperty("--chat-available-height", `${available}px`);
23 +      });
24 +    };
25 +    measure();
26 +    viewport.addEventListener("resize", measure);
27 +    viewport.addEventListener("scroll", measure);
28 +    window.addEventListener("resize", measure);
29 +    window.addEventListener("scroll", measure);
30 +    return () => {
31 +      cancelAnimationFrame(frame);
32 +      viewport.removeEventListener("resize", measure);
33 +      viewport.removeEventListener("scroll", measure);
34 +      window.removeEventListener("resize", measure);
35 +      window.removeEventListener("scroll", measure);
36 +      element.style.removeProperty("--chat-available-height");
37 +    };
38 +  }, [element]);
39 +  return setElement;
40 +}
```

```

1  import { apiErrorMessage } from "../utils/apiError.js";
2  import { useChatViewport } from "../hooks/useChatViewport";
3  import chatStyles from "../styles/chat.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import { useCallback, useEffect, useRef, useState } from "react";
7
8  import { useSearchParams } from "react-router-dom";
9
10 import Navbar from "../components/layout/Navbar";
11
12 import ConversationList from "../components/chat/ConversationList";
13 import ChatWindow from "../components/chat/ChatWindow";
14
15 import {
16   getConversations,
17   getConversation,
18   getOrCreateConversation,
19 } from "../api/conversationApi";
20
21 import { useAuth } from "../auth/AuthContext";
22
23 import { createConversationUpdateWebSocket } from "../services/conversationUpdateWebSocketService";
24 import { subscribeConversationSidebar } from "../services/conversationSidebarSubscription";
25 import { applyConversationUpdate, mergeConversation, mergeConversationLists } from "../utils/conversationUpdates"
26 > ;
27
28 const css = bindStyles(chatStyles);
29
30 const Messages = () => {
31   const viewportRef = useChatViewport();
32   const { user } = useAuth();
33
34   const [searchParams, setSearchParams] = useSearchParams();
35
36   const [conversations, setConversations] = useState([]);
37
38   const [selectedConversation, setSelectedConversation] = useState(null);
39
40   const [loading, setLoading] = useState(true);
41
42   const [error, setError] = useState("");
43   const conversationsRef = useRef(conversations);
44   useEffect(() => {
45     conversationsRef.current = conversations;
46   }, [conversations]);
47
48   /*
49    * Desktop always shows both panels.
50    *
51    * On mobile this controls whether we are currently
52    * looking at the conversation list or the chat window.
53    */
54   const [mobileView, setMobileView] = useState("conversations");
55
56   // =====
57   // LOAD CONVERSATIONS
58   // =====
59
60   const loadConversations = useCallback(async () => {
61     try {
62       setLoading(true);
63       setError("");
64
65       const data = await getConversations();
66
67       setConversations((current) => mergeConversationLists(current, data || []));
68
69       return data || [];
70     } catch (error) {

```

```

70     console.error("Failed to load conversations:", error);
71
72     setError(apiErrorMessage(error, "Unable to load conversations"));
73
74     return [];
75   } finally {
76     setLoading(false);
77   }
78 }, []);
79
80 // =====
81 // INITIAL LOAD
82 // =====
83
84 useEffect(() => {
85   let cancelled = false;
86
87   const initialize = async () => {
88     const data = await loadConversations();
89
90     if (cancelled) {
91       return;
92     }
93
94     const requestedUserId = searchParams.get("userId");
95
96     if (!requestedUserId) {
97       if (data.length > 0) {
98         setSelectedConversation((current) => current || data[0]);
99       }
100
101       return;
102     }
103
104     try {
105       const conversation = await getOrCreateConversation(requestedUserId);
106
107       if (cancelled) {
108         return;
109       }
110
111       const refreshed = await loadConversations();
112
113       if (cancelled) {
114         return;
115       }
116
117       const matchingConversation = refreshed.find(
118         (item) => String(item.id) === String(conversation.id),
119       );
120
121       const resolvedConversation = matchingConversation || conversation;
122
123       setSelectedConversation(resolvedConversation);
124
125       /*
126        * When Messages is opened directly through
127        * /messages?userId=..., mobile should go
128        * directly into the chat.
129        */
130       setMobileView("chat");
131
132       setSearchParams(
133         {},
134         {
135           replace: true,
136         },
137       );
138     } catch (error) {
139       if (!cancelled) {

```

```

140         console.error("Failed to open conversation:", error);
141
142         setError(
143             apiErrorMessage(error, "Unable to start conversation"),
144         );
145     }
146 }
147 };
148
149 initialize();
150
151 return () => {
152     cancelled = true;
153 };
154 }, [loadConversations, searchParams, setSearchParams]);
155
156 // =====
157 // SELECT CONVERSATION
158 // =====
159
160 const handleSelectConversation = (conversation) => {
161     setSelectedConversation(conversation);
162
163     /*
164      * On mobile, selecting a conversation should
165      * switch from the conversation list to chat.
166      *
167      * On desktop this state has no visual effect.
168      */
169     setMobileView("chat");
170 };
171
172 // =====
173 // BACK TO CONVERSATIONS
174 // =====
175
176 const handleBackToConversations = () => {
177     setMobileView("conversations");
178 };
179
180 // =====
181 // HANDLE REALTIME MESSAGE
182 // =====
183
184 const handleConversationUpdate = useCallback((update) => {
185     if (update?.conversationId == null || !update.lastMessage) return;
186     setConversations((current) => applyConversationUpdate(current, update));
187     setSelectedConversation((current) =>
188         current && String(current.id) === String(update.conversationId)
189             ? mergeConversation(current, {
190                 ...current,
191                 lastMessage: update.lastMessage,
192                 updatedAt: update.updatedAt || update.lastMessage.createdAt,
193             })
194             : current,
195     );
196 }, []);
197
198 const handleMessageReceived = useCallback((message) => {
199     if (message?.conversationId == null) return;
200     handleConversationUpdate({
201         conversationId: message.conversationId,
202         lastMessage: message,
203         updatedAt: message.createdAt,
204     });
205 }, [handleConversationUpdate]);
206
207 useEffect(() => {
208     if (!user?.id) return;
209     return subscribeConversationSidebar({

```

```

210     createSocket: createConversationUpdateWebSocket,
211     loadList: getConversations,
212     loadConversation: getConversation,
213     hasConversation: (id) =>
214       conversationsRef.current.some((row) => String(row.id) === id),
215     onUpdate: handleConversationUpdate,
216     onRows: (rows) => {
217       setConversations((current) => mergeConversationLists(current, rows));
218       setSelectedConversation((current) => {
219         const incoming = rows.find((row) => String(row.id) === String(current?.id));
220         return current && incoming ? mergeConversation(current, incoming) : current;
221       });
222     },
223     onError: (error) => {
224       console.error("Conversation sidebar update failed:", error);
225     },
226   });
227 }, [user?.id, handleConversationUpdate]);
228
229 // =====
230 // LOADING
231 // =====
232
233 if (loading && conversations.length === 0) {
234   return (
235     <>
236       <Navbar />
237
238       <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Loading messages...<
239     </div>
240     </>
241   );
242 }
243
244 return (
245   <>
246     <Navbar />
247
248     <main ref={viewportRef} id="main-content" tabIndex={-1} className={css("messages-page")}>
249       <div className={css("messages-card")}>
250         {error && <StatusMessage tone="error" className={css("error chat-page-error")}><error></StatusMessage>}
251
252         <div
253           className={css(`messages-layout ${
254             mobileView === "chat"
255               ? "mobile-chat-active"
256               : "mobile-conversations-active"
257           })}
258         >
259           <ConversationList
260             conversations={conversations}
261             selectedConversationId={selectedConversation?.id}
262             onSelect={handleSelectConversation}
263             loading={loading}
264           />
265
266           <ChatWindow
267             conversation={selectedConversation}
268             currentUserId={user?.id}
269             onMessageReceived={handleMessageReceived}
270             onBackToConversations={handleBackToConversations}
271           />
272         </div>
273       </div>
274     </main>
275   </>
276 );
277
278 export default Messages;

```

src/pages/Messages.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/Messages.jsx
2  +++ after/src/pages/Messages.jsx
3  @@ -1,4 +1,5 @@
4  import { apiErrorMessage } from "../utils/apiError.js";
5  +import { useChatViewport } from "../hooks/useChatViewport";
6  import chatStyles from "../styles/chat.module.css";
7  import { bindStyles } from "../utils/bindStyles";
8  import StatusMessage from "../components/ui/StatusMessage";
9  @@ -26,6 +27,7 @@
10 const css = bindStyles(chatStyles);
11
12 const Messages = () => {
13 + const viewportRef = useChatViewport();
14   const { user } = useAuth();
15
16   const [searchParams, setSearchParams] = useSearchParams();
17  @@ -242,7 +244,7 @@
18     <>
19       <Navbar />
20
21 -     <main id="main-content" tabIndex={-1} className={css("messages-page")}>
22 +     <main ref={viewportRef} id="main-content" tabIndex={-1} className={css("messages-page")}>
23       <div className={css("messages-card")}>
24         {error && <StatusMessage tone="error" className={css("error chat-page-error")}>{error}</StatusMessage>
25       > }
26     </>
27   </>
28 }
```

```

1  import { formatTimestamp, notificationLabel } from "../utils/displayText.js";
2  import { apiErrorMessage } from "../utils/apiError.js";
3  import notificationsStyles from "../styles/notifications.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import PageContainer from "../components/ui/PageContainer";
7  import Card from "../components/ui/Card";
8  import { useEffect, useState } from "react";
9
10 import Navbar from "../components/layout/Navbar";
11
12 import { useNotifications } from "../context/NotificationContext";
13
14 const css = bindStyles(notificationsStyles);
15
16 const PAGE_SIZE = 10;
17
18 const Notifications = () => {
19   const {
20     notifications,
21     loading,
22     error,
23     loadNotifications,
24     markAsRead,
25     markAllAsRead,
26   } = useNotifications();
27
28   const [page, setPage] = useState(0);
29
30   const [totalPages, setTotalPages] = useState(0);
31
32   const [actionLoadingId, setActionLoadingId] = useState(null);
33
34   const [markAllLoading, setMarkAllLoading] = useState(false);
35
36   const [actionError, setActionError] = useState("");
37
38   // =====
39   // LOAD PAGE
40   // =====
41
42   const loadPage = async (nextPage) => {
43     try {
44       setActionError("");
45
46       const data = await loadNotifications(nextPage, PAGE_SIZE);
47
48       setPage(data?.number ?? nextPage);
49       setTotalPages(data?.totalPages ?? 0);
50     } catch (error) {
51       console.error("Failed to load notification page:", error);
52     }
53   };
54
55   // =====
56   // INITIAL LOAD
57   // =====
58
59   useEffect(() => {
60     loadPage(0);
61   }, []);
62
63   // =====
64   // MARK ONE AS READ
65   // =====
66
67   const handleMarkAsRead = async (notificationId) => {
68     try {
69       setActionError("");
70       setActionLoadingId(notificationId);

```

```

71
72     await markAsRead(notificationId);
73   } catch (error) {
74     console.error("Failed to mark notification as read:", error);
75
76     setActionError(
77       apiErrorMessage(error, "Unable to mark notification as read"),
78     );
79   } finally {
80     setActionLoadingId(null);
81   }
82 };
83
84 // =====
85 // MARK ALL AS READ
86 // =====
87
88 const handleMarkAllAsRead = async () => {
89   try {
90     setActionError("");
91     setMarkAllLoading(true);
92
93     await markAllAsRead();
94   } catch (error) {
95     console.error("Failed to mark all notifications as read:", error);
96
97     setActionError(
98       apiErrorMessage(error, "Unable to mark all notifications as read"),
99     );
100   } finally {
101     setMarkAllLoading(false);
102   }
103 };
104
105 // =====
106 // LOADING
107 // =====
108
109 if (loading && notifications.length === 0) {
110   return (
111     <>
112       <Navbar />
113
114       <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Loading notification
115     </div>
116   );
117 }
118
119 // =====
120 // UI
121 // =====
122
123 return (
124   <>
125     <Navbar />
126
127     <PageContainer className={css("notifications-page")}>
128       <Card className={css("notifications-card")}>
129         <div className={css("notifications-header")}>
130           <div>
131             <h1>Notifications</h1>
132
133             <p>Stay up to date with your FrndBook activity.</p>
134           </div>
135
136           {notifications.some((notification) => !notification.read) && (
137             <button
138               type="button"
139               className={css("notifications-mark-all")}

```

```

140         onClick={handleMarkAllAsRead}
141         disabled={markAllLoading}
142       >
143         {markAllLoading ? "Marking..." : "Mark all as read"}
144       </button>
145     )}
146   </div>
147
148   {(error || actionError) && (
149     <StatusMessage tone="error" className={css("error")}>{actionError || error}</StatusMessage>
150   )}
151
152   {notifications.length === 0 && !error && (
153     <div className={css("notifications-empty")}>
154       <h2>No notifications yet</h2>
155
156       <p>You're all caught up. New activity will appear here.</p>
157     </div>
158   )}
159
160   {notifications.length > 0 && (
161     <div className={css("notification-list")}>
162       {notifications.map((notification) => (
163         <article
164           key={notification.id}
165           className={css(`notification-item ${
166             notification.read
167               ? "notification-read"
168               : "notification-unread"
169           })`}
170         >
171           <div className={css("notification-indicator")}>
172             {!notification.read && (
173               <span
174                 className={css("notification-unread-dot")}
175                 aria-label="Unread"
176               />
177             )}
178           </div>
179
180           <div className={css("notification-content")}>
181             <p className={css("notification-message")}>
182               {notification.message}
183             </p>
184
185             <div className={css("notification-meta")}>
186               <span>
187                 {notification.createdAt
188                   ? formatTimestamp(notification.createdAt)
189                   : ""}
190               </span>
191
192               {notification.type && (
193                 <span className={css("notification-type")}>
194                   {notificationLabel(notification.type)}
195                 </span>
196               )}
197             </div>
198           </div>
199
200           {!notification.read && (
201             <button
202               type="button"
203               className={css("notification-read-button")}
204               onClick={() => handleMarkAsRead(notification.id)}
205               disabled={actionLoadingId === notification.id}
206             >
207               {actionLoadingId === notification.id
208                 ? "..."
209                 : "Mark read"}

```

```
210             </button>
211         )}
212     </article>
213   )}
214 </div>
215 )}
216
217 {totalPages > 1 && (
218   <div className={css("notification-pagination")}>
219     <button
220       type="button"
221       disabled={loading || page === 0}
222       onClick={() => loadPage(page - 1)}
223     >
224       Previous
225     </button>
226
227     <span>
228       Page {page + 1} of {totalPages}
229     </span>
230
231     <button
232       type="button"
233       disabled={loading || page >= totalPages - 1}
234       onClick={() => loadPage(page + 1)}
235     >
236       Next
237     </button>
238   </div>
239 )}
240 </Card>
241 </PageContainer>
242 </>
243 );
244 };
245
246 export default Notifications;
```

src/pages/Notifications.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- before/src/pages/Notifications.jsx
2 +++ after/src/pages/Notifications.jsx
3 @@ -1,3 +1,4 @@
4 +import { formatTimestamp, notificationLabel } from "../utils/displayText.js";
5 import { apiErrorMessage } from "../utils/apiError.js";
6 import notificationsStyles from "../styles/notifications.module.css";
7 import { bindStyles } from "../utils/bindStyles";
8 @@ -184,13 +185,13 @@
9         <div className={css("notification-meta")}>
10             <span>
11                 {notification.createdAt
12 -                 ? new Date(notification.createdAt).toLocaleString()
13 +                 ? formatTimestamp(notification.createdAt)
14                 : ""}
15             </span>
16
17             {notification.type && (
18                 <span className={css("notification-type")}>
19 -                 {notification.type}
20 +                 {notificationLabel(notification.type)}
21             </span>
22             )}
23         </div>
```

```

1  import { formatTimestamp } from "../utils/displayText.js";
2  import { apiError } from "../utils/apiError";
3  import { apiErrorMessage } from "../utils/apiError.js";
4  import profileStyles from "../styles/profile.module.css";
5  import { bindStyles } from "../utils/bindStyles";
6  import StatusMessage from "../components/ui/StatusMessage";
7  import FormField from "../components/ui/FormField";
8  import PageContainer from "../components/ui/PageContainer";
9  import Card from "../components/ui/Card";
10 import { useEffect, useRef, useState } from "react";
11
12 import Navbar from "../components/layout/Navbar";
13
14 import UserAvatar from "../components/users/UserAvatar";
15
16 import { updateProfile, updateProfileImage } from "../api/userApi";
17
18 import { useAuth } from "../auth/AuthContext";
19
20 const css = bindStyles(profileStyles);
21
22 const ProfileForm = () => {
23   const { user, loading: authLoading, updateUser, sessionId } = useAuth();
24   const profile = user;
25   const busy = useRef(false);
26   const mounted = useRef(false);
27   useEffect(() => { mounted.current = true; return () => { mounted.current = false; }; }, []);
28
29   const [nameDraft, setName] = useState(null);
30   const name = nameDraft ?? user?.name ?? "";
31
32   const [bioDraft, setBio] = useState(null);
33   const bio = bioDraft ?? user?.bio ?? "";
34
35   const [loading, setLoading] = useState(false);
36
37   const [imageLoading, setImageLoading] = useState(false);
38
39   const [error, setError] = useState("");
40   const [fieldErrors, setFieldErrors] = useState({});
41
42   const [success, setSuccess] = useState("");
43
44   const [previewImage, setPreviewImage] = useState(null);
45
46   const fileInputRef = useRef(null);
47
48   // =====
49   // CLEANUP PREVIEW
50   // =====
51
52   useEffect(() => {
53     return () => {
54       if (previewImage) {
55         URL.revokeObjectURL(previewImage);
56       }
57     };
58   }, [previewImage]);
59
60   // =====
61   // UPDATE PROFILE
62   // =====
63
64   const handleSubmit = async (event) => {
65     event.preventDefault();
66     if (busy.current) return;
67     busy.current = true;
68
69     setError("");
70     setFieldErrors({});

```

```

71     setSuccess("");
72     setLoading(true);
73
74     try {
75         const updatedUser = await updateProfile(name.trim(), bio.trim());
76
77         if (!mounted.current || !updateUser(updatedUser, sessionKey)) return;
78         setName(null);
79         setBio(null);
80
81         setSuccess("Profile updated successfully.");
82     } catch (error) {
83         if (!mounted.current) return;
84         setFieldErrors(apiErrorMessage(error).fieldErrors);
85         console.error("Profile update failed:", error);
86
87         setError(apiErrorMessage(error, "Failed to update profile"));
88     } finally {
89         busy.current = false;
90         if (mounted.current) setLoading(false);
91     }
92 };
93
94 // =====
95 // IMAGE SELECTION
96 // =====
97
98 const handleImageChange = (event) => {
99     if (busy.current) return;
100     const file = event.target.files?.[0];
101
102     if (!file) {
103         return;
104     }
105
106     if (!["image/jpeg", "image/png", "image/webp"].includes(file.type)) {
107         setError("Please select a JPEG, PNG or WebP image.");
108         return;
109     }
110
111     setError("");
112     setFieldErrors({});
113     setSuccess("");
114
115     if (previewImage) {
116         URL.revokeObjectURL(previewImage);
117     }
118
119     const previewUrl = URL.createObjectURL(file);
120
121     setPreviewImage(previewUrl);
122
123     void handleImageUpload(file);
124 };
125
126 // =====
127 // IMAGE UPLOAD
128 // =====
129
130 const handleImageUpload = async (file) => {
131     busy.current = true;
132     setImageLoading(true);
133
134     setError("");
135     setFieldErrors({});
136     setSuccess("");
137
138     try {
139         const updatedUser = await updateProfileImage(file);
140

```

```

141     if (!mounted.current || !updateUser(updatedUser, sessionKey)) return;
142
143     setSuccess("Profile image updated successfully.");
144
145     setPreviewImage(null);
146   } catch (error) {
147     if (!mounted.current) return;
148     setFieldErrors(apiError(error).fieldErrors);
149     console.error("Profile image upload failed:", error);
150
151     if (previewImage) {
152       URL.revokeObjectURL(previewImage);
153     }
154
155     setPreviewImage(null);
156
157     setError(
158       apiErrorMessage(error, "Failed to upload profile image"),
159     );
160   } finally {
161     busy.current = false;
162     if (mounted.current) setImageLoading(false);
163
164     if (fileInputRef.current) {
165       fileInputRef.current.value = "";
166     }
167   }
168 };
169
170 // =====
171 // LOADING
172 // =====
173
174 if (authLoading) {
175   return <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Loading...</div>;
176 }
177
178 if (!profile) {
179   return <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Unable to load pr
180   > ofile.</div>;
181 }
182
183 return (
184   <>
185     <Navbar />
186
187     <PageContainer className={css("profile-page")}>
188       <Card className={css("profile-card")}>
189         <h1>My Profile</h1>
190
191         {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
192
193         {success && <StatusMessage tone="success" className={css("success")}>{success}</StatusMessage>}
194
195         <div className={css("profile-image-section")}>
196           <UserAvatar
197             name={profile.name}
198             image={previewImage || profile.profileImage}
199             userId={profile.id}
200             size="large"
201           />
202
203           <button
204             type="button"
205             onClick={() => fileInputRef.current?.click()}
206             disabled={imageLoading || loading}
207           >
208             {imageLoading ? "Uploading..." : "Change Photo"}
209           </button>

```

```

210         <input
211             ref={fileInputRef}
212             type="file"
213             accept="image/jpeg,image/png,image/webp"
214             onChange={handleImageChange}
215             hidden
216         />
217     </div>
218
219     <form className={css("profile-form")} onSubmit={handleSubmit}>
220
221         <FormField label="Name" error={fieldErrors.name} autoComplete="name"
222             type="text"
223             disabled={loading || imageLoading}
224             value={name}
225             maxLength={100}
226             onChange={(event) => setName(event.target.value)}
227             required
228         />
229
230         <FormField label="Email" type="email" autoComplete="email" value={profile.email} disabled />
231
232         <FormField as="textarea" label="Bio" error={fieldErrors.bio}
233             disabled={loading || imageLoading}
234             value={bio}
235             maxLength={500}
236             rows={5}
237             onChange={(event) => setBio(event.target.value)}
238         />
239
240         <div className={css("character-count")}>{bio.length}/500</div>
241
242         <button type="submit" disabled={loading || imageLoading}>
243             {loading ? "Saving..." : "Save Changes"}
244         </button>
245     </form>
246
247     <div className={css("profile-details")}>
248         <p>
249             <strong>Status:</strong> {profile.status || "-"}
250         </p>
251
252         <p>
253             <strong>Last seen:</strong>{" "}
254             {profile.lastSeen
255                 ? formatTimestamp(profile.lastSeen)
256                 : "-"}
257         </p>
258     </div>
259 </Card>
260 </PageContainer>
261 </>
262 );
263 };
264
265 export default function Profile() {
266     const { user, sessionKey } = useAuth();
267     return <ProfileForm key={` ${sessionKey}:${user?.id}`} />;
268 }

```

src/pages/Profile.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- before/src/pages/Profile.jsx
2 +++ after/src/pages/Profile.jsx
3 @@ -1,3 +1,4 @@
4 +import { formatTimestamp } from "../utils/displayText.js";
5 import { apiError } from "../utils/apiError";
6 import { apiErrorMessage } from "../utils/apiError.js";
7 import profileStyles from "../styles/profile.module.css";
8 @@ -251,7 +252,7 @@
9         <p>
10             <strong>Last seen:</strong>{" "}
11             {profile.lastSeen
12 -             ? new Date(profile.lastSeen).toLocaleString()
13 +             ? formatTimestamp(profile.lastSeen)
14             : "-"}
15         </p>
16     </div>
```

```

1  import { formatTimestamp } from "../utils/displayText.js";
2  import { apiErrorMessage } from "../utils/apiError.js";
3  import friendsStyles from "../styles/friends.module.css";
4  import profileStyles from "../styles/profile.module.css";
5  import { bindStyles } from "../utils/bindStyles";
6  import StatusMessage from "../components/ui/StatusMessage";
7  import PageContainer from "../components/ui/PageContainer";
8  import Card from "../components/ui/Card";
9  import { useEffect, useState } from "react";
10 import { Link, useNavigate, useParams } from "react-router-dom";
11
12 import Navbar from "../components/layout/Navbar";
13
14 import UserAvatar from "../components/users/UserAvatar";
15 import FriendAction from "../components/users/FriendAction";
16
17 import { getUserById } from "../api/userApi";
18
19 import {
20   sendFriendRequest,
21   getFriends,
22   getReceivedFriendRequests,
23   getSentFriendRequests,
24   acceptFriendRequest,
25   rejectFriendRequest,
26   removeFriend,
27 } from "../api/friendApi";
28
29 import { useAuth } from "../auth/AuthContext";
30
31 import { FRIENDSHIP_STATUS, getFriendshipState } from "../utils/friendship";
32
33 const css = bindStyles(friendsStyles, profileStyles);
34
35 const UserProfile = () => {
36   const { userId } = useParams();
37
38   const navigate = useNavigate();
39
40   const { user: currentUser } = useAuth();
41
42   const [user, setUser] = useState(null);
43
44   const [friendshipStatus, setFriendshipStatus] = useState(
45     FRIENDSHIP_STATUS.NONE,
46   );
47
48   const [friendRequestId, setFriendRequestId] = useState(null);
49
50   const [loading, setLoading] = useState(true);
51
52   const [friendshipLoading, setFriendshipLoading] = useState(false);
53
54   const [friendshipLoaded, setFriendshipLoaded] = useState(false);
55
56   const [error, setError] = useState("");
57
58   const [friendshipError, setFriendshipError] = useState("");
59
60   // =====
61   // LOAD PROFILE
62   // =====
63
64   useEffect(() => {
65     const loadUser = async () => {
66       try {
67         setLoading(true);
68         setError("");
69
70         const data = await getUserById(userId);

```

```

71
72     setUser(data);
73   } catch (error) {
74     console.error("Failed to load user profile:", error);
75
76     setError(
77       apiErrorMessage(error, "Unable to load user profile"),
78     );
79   } finally {
80     setLoading(false);
81   }
82 };
83
84   loadUser();
85 }, [userId]);
86
87 // =====
88 // LOAD FRIENDSHIP STATE
89 // =====
90
91 useEffect(() => {
92   if (!user || !currentUser) {
93     return;
94   }
95
96   if (String(user.id) === String(currentUser.id)) {
97     setFriendshipLoaded(true);
98
99     return;
100   }
101
102   const loadFriendshipState = async () => {
103     try {
104       setFriendshipError("");
105       setFriendshipLoaded(false);
106
107       const [friends, receivedRequests, sentRequests] = await Promise.all([
108         getFriends(),
109         getReceivedFriendRequests(),
110         getSentFriendRequests(),
111       ]);
112
113       const state = getFriendshipState(
114         user.id,
115         friends || [],
116         receivedRequests || [],
117         sentRequests || [],
118       );
119
120       setFriendshipStatus(state.status);
121
122       setFriendRequestId(state.requestId);
123     } catch (error) {
124       console.error("Failed to load friendship state:", error);
125
126       setFriendshipError(
127         apiErrorMessage(error, "Unable to load friendship status"),
128       );
129     } finally {
130       setFriendshipLoaded(true);
131     }
132   };
133
134   loadFriendshipState();
135 }, [user, currentUser]);
136
137 // =====
138 // ADD FRIEND
139 // =====
140

```

```

141   const handleAddFriend = async () => {
142     if (!user) {
143       return;
144     }
145
146     try {
147       setFriendshipLoading(true);
148       setFriendshipError("");
149
150       const request = await sendFriendRequest(user.id);
151
152       setFriendshipStatus(FRIENDSHIP_STATUS.REQUEST_SENT);
153
154       setFriendRequestId(request?.id || null);
155     } catch (error) {
156       console.error("Failed to send friend request:", error);
157
158       setFriendshipError(
159         apiErrorMessage(error, "Unable to send friend request"),
160       );
161     } finally {
162       setFriendshipLoading(false);
163     }
164   };
165
166   // =====
167   // ACCEPT
168   // =====
169
170   const handleAcceptFriend = async () => {
171     if (!friendRequestId) {
172       return;
173     }
174
175     try {
176       setFriendshipLoading(true);
177       setFriendshipError("");
178
179       await acceptFriendRequest(friendRequestId);
180
181       setFriendshipStatus(FRIENDSHIP_STATUS.FRIENDS);
182
183       setFriendRequestId(null);
184     } catch (error) {
185       console.error("Failed to accept friend request:", error);
186
187       setFriendshipError(
188         apiErrorMessage(error, "Unable to accept friend request"),
189       );
190     } finally {
191       setFriendshipLoading(false);
192     }
193   };
194
195   // =====
196   // REJECT
197   // =====
198
199   const handleRejectFriend = async () => {
200     if (!friendRequestId) {
201       return;
202     }
203
204     try {
205       setFriendshipLoading(true);
206       setFriendshipError("");
207
208       await rejectFriendRequest(friendRequestId);
209
210       setFriendshipStatus(FRIENDSHIP_STATUS.NONE);

```

```

211         setFriendRequestId(null);
212     } catch (error) {
213         console.error("Failed to reject friend request:", error);
214     }
215     setFriendshipError(
216         apiErrorMessage(error, "Unable to reject friend request"),
217     );
218 } finally {
219     setFriendshipLoading(false);
220 }
221 };
222
223 // =====
224 // REMOVE FRIEND
225 // =====
226
227 const handleRemoveFriend = async () => {
228     if (!user) {
229         return;
230     }
231
232     const confirmed = window.confirm(
233         `Are you sure you want to remove ${user.name} from your friends?`,
234     );
235
236     if (!confirmed) {
237         return;
238     }
239
240     try {
241         setFriendshipLoading(true);
242         setFriendshipError("");
243
244         await removeFriend(user.id);
245
246         setFriendshipStatus(FRIENDSHIP_STATUS.NONE);
247     } catch (error) {
248         console.error("Failed to remove friend:", error);
249
250         setFriendshipError(
251             apiErrorMessage(error, "Unable to remove friend"),
252         );
253     } finally {
254         setFriendshipLoading(false);
255     }
256 }
257 };
258
259 // =====
260 // OPEN MESSAGE
261 // =====
262
263 const handleMessage = () => {
264     if (!user) {
265         return;
266     }
267
268     /*
269     * Reuse the existing Messages page flow.
270     *
271     * Messages.jsx already handles:
272     *
273     * /messages?userId={userId}
274     *
275     * and resolves/opens the conversation using
276     * getOrCreateConversation().
277     */
278     navigate(`/messages?userId=${user.id}`);
279 };
280

```

```

281 // =====
282 // LOADING
283 // =====
284
285 if (loading) {
286   return (
287     <>
288       <Navbar />
289
290       <div id="main-content" tabIndex=-1 role="status" className={css("loading-screen")}>Loading profile...</div>
291     </>
292   );
293 }
294
295 // =====
296 // PROFILE ERROR
297 // =====
298
299 if (error || !user) {
300   return (
301     <>
302       <Navbar />
303
304       <PageContainer className={css("profile-page")}>
305         <Card className={css("profile-card")}>
306           <StatusMessage tone="error" className={css("error")}>{error || "User not found."}</StatusMessage>
307
308           <Link to="/" className={css("back-link")}>
309             ← Back to Home
310           </Link>
311         </Card>
312       </PageContainer>
313     </>
314   );
315 }
316
317 const isOwnProfile = String(user.id) === String(currentUser?.id);
318
319 // =====
320 // UI
321 // =====
322
323 return (
324   <>
325     <Navbar />
326
327     <PageContainer className={css("profile-page")}>
328       <Card className={css("profile-card user-profile-card")}>
329         <Link to="/" className={css("back-link")}>
330           ← Back to Home
331         </Link>
332
333         <div className={css("profile-image-section")}>
334           <UserAvatar
335             name={user.name}
336             image={user.profileImage}
337             userId={user.id}
338             size="large"
339           />
340         </div>
341
342         <div className={css("user-profile-info")}>
343           <h1>{user.name}</h1>
344
345           <p className={css("user-profile-bio")}>{user.bio || "No bio available"}</p>
346
347           <div className={css("profile-details")}>
348             <p>
349               <strong>Status:</strong> {user.status || "-"}

```

```

350         </p>
351
352         <p>
353             <strong>Last seen:</strong>{" "}
354             {user.lastSeen ? formatTimestamp(user.lastSeen) : "-"}
355         </p>
356     </div>
357 </div>
358
359     {!isOwnProfile && (
360     <div className={css("friend-action-section")}>
361         {friendshipError && <StatusMessage tone="error" className={css("error")}>{friendshipError}</StatusM
362     > message>}
363
364         {!friendshipLoaded ? (
365             <p className={css("friendship-loading")}>Checking friendship...</p>
366         ) : (
367             <FriendAction
368                 status={friendshipStatus}
369                 onAdd={handleAddFriend}
370                 onAccept={handleAcceptFriend}
371                 onReject={handleRejectFriend}
372                 onRemove={handleRemoveFriend}
373                 onMessage={handleMessage}
374                 loading={friendshipLoading}
375             />
376         )}
377     </div>
378     )}
379 </Card>
380 </PageContainer>
381 </>
382 );
383 };
384 export default UserProfile;

```

src/pages/UserProfile.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/UserProfile.jsx
2  +++ after/src/pages/UserProfile.jsx
3  @@ -1,3 +1,4 @@
4  +import { formatTimestamp } from "../utils/displayText.js";
5  import { apiErrorMessage } from "../utils/apiError.js";
6  import friendsStyles from "../styles/friends.module.css";
7  import profileStyles from "../styles/profile.module.css";
8  @@ -350,7 +351,7 @@
9
10         <p>
11             <strong>Last seen:</strong>{" "}
12 -             {user.lastSeen ? new Date(user.lastSeen).toLocaleString() : "-"}
13 +             {user.lastSeen ? formatTimestamp(user.lastSeen) : "-"}
14         </p>
15     </div>
16 </div>
```

```
1  .messages-page {
2    width: 100%;
3    max-width: 1200px;
4    height: var(--chat-available-height, calc(100dvh - var(--navbar-height)));
5    margin: 0 auto;
6    padding: 24px 20px;
7    box-sizing: border-box;
8  }
9
10 .messages-card {
11   width: 100%;
12   height: 100%;
13   min-height: 0;
14   background: var(--color-surface);
15   border-radius: 12px;
16   box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
17   overflow: hidden;
18   display: flex;
19   flex-direction: column;
20 }
21
22 .messages-layout {
23   flex: 1;
24   min-width: 0;
25   min-height: 0;
26   display: grid;
27   grid-template-columns: 320px minmax(0, 1fr);
28 }
29
30 .conversation-list {
31   min-width: 0;
32   min-height: 0;
33   border-right: 1px solid var(--color-item-border);
34   display: flex;
35   flex-direction: column;
36   overflow: hidden;
37 }
38
39 .conversation-list-header {
40   flex-shrink: 0;
41   padding: 18px 20px;
42   border-bottom: 1px solid var(--color-item-border);
43 }
44
45 .conversation-list-header h2 {
46   margin: 0;
47   font-size: 18px;
48 }
49
50 .conversation-items {
51   flex: 1;
52   min-width: 0;
53   min-height: 0;
54   overflow-y: auto;
55   overflow-x: hidden;
56   overscroll-behavior: contain;
57 }
58
59 .conversation-item {
60   width: 100%;
61   min-height: 72px;
62   box-sizing: border-box;
63   padding: 12px 14px;
64   border: none;
65   border-bottom: 1px solid var(--color-border-light);
66   background: var(--color-surface);
67   display: flex;
68   align-items: center;
69   gap: 12px;
70   text-align: left;
```

```
71   cursor: pointer;
72 }
73
74 .conversation-item:hover {
75   background: var(--color-hover);
76 }
77
78 .conversation-item-active {
79   background: var(--color-selected);
80 }
81
82 .conversation-item-content {
83   min-width: 0;
84   flex: 1;
85   display: flex;
86   flex-direction: column;
87   gap: 4px;
88 }
89
90 .conversation-item-content strong {
91   overflow: hidden;
92   white-space: nowrap;
93   text-overflow: ellipsis;
94 }
95
96 .conversation-item-content span {
97   color: var(--color-text-secondary);
98   font-size: 13px;
99   overflow: hidden;
100  white-space: nowrap;
101  text-overflow: ellipsis;
102 }
103
104 .conversation-item-time {
105   color: var(--color-text-muted);
106   font-size: 11px;
107   flex-shrink: 0;
108 }
109
110 .conversation-empty {
111   padding: 25px 20px;
112   color: var(--color-text-secondary);
113   text-align: center;
114 }
115
116 .chat-status {
117   flex-shrink: 0;
118   padding: 15px;
119   color: var(--color-text-subtle);
120 }
121
122 .chat-window {
123   min-width: 0;
124   min-height: 0;
125   overflow: hidden;
126   display: flex;
127   flex-direction: column;
128 }
129
130 .chat-window-empty {
131   align-items: center;
132   justify-content: center;
133   color: var(--color-text-subtle);
134 }
135
136 .chat-header {
137   flex-shrink: 0;
138   min-height: 72px;
139   padding: 12px 18px;
140   border-bottom: 1px solid var(--color-item-border);
```

```
141   display: flex;
142   align-items: center;
143   gap: 12px;
144   box-sizing: border-box;
145 }
146
147 .chat-header h2 {
148   margin: 0 0 4px;
149   font-size: 18px;
150   overflow: hidden;
151   text-overflow: ellipsis;
152   white-space: nowrap;
153 }
154
155 .chat-header-avatar-link {
156   display: flex;
157   flex-shrink: 0;
158   border-radius: 50%;
159   text-decoration: none;
160 }
161
162 .chat-header-avatar-link:focus-visible {
163   outline: 2px solid var(--color-outline);
164   outline-offset: 2px;
165 }
166
167 .chat-connection-status {
168   color: var(--color-text-muted);
169   font-size: 12px;
170 }
171
172 .chat-connection-status.connected {
173   color: var(--color-success);
174 }
175
176 .chat-header-info {
177   min-width: 0;
178   flex: 1;
179 }
180
181 .mobile-chat-back-button {
182   display: none;
183   flex-shrink: 0;
184   width: 38px;
185   height: 38px;
186   padding: 0;
187   border: 1px solid #dddddd;
188   border-radius: 50%;
189   background: var(--color-surface);
190   color: var(--color-text);
191   font-size: 20px;
192   align-items: center;
193   justify-content: center;
194   cursor: pointer;
195 }
196
197 .mobile-chat-back-button:hover {
198   background: var(--color-hover);
199 }
200
201 .chat-error {
202   flex-shrink: 0;
203   margin: 10px 15px;
204 }
205
206 .chat-page-error {
207   flex-shrink: 0;
208   margin: 12px 15px;
209 }
210
```

```
211 .message-list-container {
212   flex: 1;
213   min-width: 0;
214   min-height: 0;
215   overflow: hidden;
216   display: flex;
217   flex-direction: column;
218 }
219
220 .older-messages-bar {
221   flex-shrink: 0;
222   padding: 8px 12px;
223   border-bottom: 1px solid var(--color-item-border);
224   display: flex;
225   justify-content: center;
226   align-items: center;
227   background: var(--color-surface);
228 }
229
230 .chat-secondary-button {
231   min-height: 36px;
232   padding: 7px 12px;
233   border: 1px solid var(--color-control-border);
234   border-radius: 6px;
235   background: var(--color-surface);
236   color: var(--color-text-strong);
237   cursor: pointer;
238   white-space: nowrap;
239 }
240
241 .chat-secondary-button:hover:not(:disabled) {
242   background: var(--color-hover);
243 }
244
245 .chat-secondary-button:disabled {
246   opacity: 0.6;
247 }
248
249 .message-list {
250   flex: 1;
251   min-width: 0;
252   min-height: 0;
253   padding: 20px;
254   overflow-y: auto;
255   overflow-x: hidden;
256   display: flex;
257   flex-direction: column;
258   gap: 10px;
259   box-sizing: border-box;
260   overscroll-behavior: contain;
261   scroll-behavior: smooth;
262 }
263
264 .message-list-loading {
265   align-items: center;
266   justify-content: center;
267   color: var(--color-text-subtle);
268 }
269
270 .message-empty {
271   flex: 1;
272   min-height: 100%;
273   display: flex;
274   flex-direction: column;
275   align-items: center;
276   justify-content: center;
277   color: var(--color-text-subtle);
278   text-align: center;
279 }
280
```

```
281 .message-empty p {
282   margin: 4px;
283 }
284
285 .message-row {
286   display: flex;
287   align-items: flex-end;
288   gap: 8px;
289   max-width: 75%;
290   flex-shrink: 0;
291 }
292
293 .message-row-own {
294   align-self: flex-end;
295   justify-content: flex-end;
296 }
297
298 .message-row-other {
299   align-self: flex-start;
300 }
301
302 .message-bubble {
303   min-width: 50px;
304   max-width: 100%;
305   padding: 9px 12px;
306   border-radius: 12px;
307   box-sizing: border-box;
308 }
309
310 .message-bubble p {
311   margin: 0;
312   white-space: pre-wrap;
313   overflow-wrap: anywhere;
314   word-break: break-word;
315 }
316
317 .message-bubble span {
318   display: block;
319   margin-top: 5px;
320   font-size: 10px;
321   text-align: right;
322 }
323
324 .message-bubble-own {
325   background: var(--color-primary);
326   color: var(--color-on-primary);
327   border-bottom-right-radius: 4px;
328 }
329
330 .message-bubble-other {
331   background: var(--color-selected);
332   color: var(--color-text);
333   border-bottom-left-radius: 4px;
334 }
335
336 .message-bubble-own span {
337   color: var(--color-control-border);
338 }
339
340 .message-bubble-other span {
341   color: var(--color-text-subtle);
342 }
343
344 .message-composer {
345   flex-shrink: 0;
346   border-top: 1px solid var(--color-item-border);
347   padding: 12px;
348   padding-bottom: max(12px, env(safe-area-inset-bottom));
349   display: flex;
350   flex-direction: column;
```

```
351   gap: 8px;
352   box-sizing: border-box;
353   background: var(--color-surface);
354 }
355
356 .message-composer textarea {
357   box-sizing: border-box;
358   resize: vertical;
359   min-height: 50px;
360   max-height: 150px;
361   padding: 10px;
362   border-radius: 7px;
363 }
364
365 .message-composer textarea:focus {
366   border-color: var(--color-outline);
367 }
368
369 .message-composer-footer {
370   display: flex;
371   align-items: center;
372   justify-content: space-between;
373   gap: 10px;
374 }
375
376 .message-composer-footer span {
377   color: var(--color-text-muted);
378   font-size: 12px;
379 }
380
381 .message-composer-footer button {
382   flex-shrink: 0;
383   min-height: 40px;
384   padding: 8px 18px;
385   border: none;
386   border-radius: 6px;
387   background: var(--color-primary);
388   color: var(--color-on-primary);
389   cursor: pointer;
390   font-weight: 600;
391 }
392
393 .message-composer-footer button:disabled {
394   opacity: 0.5;
395 }
396
397 @media (max-width: 800px) {
398   .messages-page {
399     height: var(--chat-available-height, calc(100dvh - var(--navbar-height)));
400     padding: 10px;
401   }
402
403   .messages-layout {
404     grid-template-columns: 1fr;
405   }
406
407   .messages-layout.mobile-conversations-active .chat-window {
408     display: none;
409   }
410
411   .messages-layout.mobile-chat-active .conversation-list {
412     display: none;
413   }
414
415   .conversation-list {
416     min-width: 0;
417     min-height: 0;
418   }
419
420   .chat-window {
```

```
421     min-width: 0;
422     min-height: 0;
423   }
424
425   .mobile-chat-back-button {
426     display: inline-flex;
427   }
428
429   .message-row {
430     max-width: 90%;
431   }
432 }
433
434 @media (max-width: 600px) {
435   .messages-page {
436     height: var(--chat-available-height, calc(100dvh - var(--navbar-height)));
437     padding: 0;
438   }
439
440   .messages-card {
441     border-radius: 0;
442     box-shadow: none;
443   }
444
445   .conversation-list-header {
446     padding: 16px;
447   }
448
449   .conversation-item {
450     min-height: 68px;
451     padding: 11px 12px;
452   }
453
454   .chat-header {
455     min-height: 64px;
456     padding: 10px 12px;
457     gap: 10px;
458   }
459
460   .chat-header h2 {
461     font-size: 16px;
462   }
463
464   .mobile-chat-back-button {
465     width: 36px;
466     height: 36px;
467   }
468
469   .message-list {
470     padding: 12px;
471     gap: 8px;
472   }
473
474   .older-messages-bar {
475     padding: 7px 10px;
476   }
477
478   .message-row {
479     max-width: 92%;
480   }
481
482   .message-bubble {
483     padding: 8px 11px;
484   }
485
486   .message-composer {
487     padding: 10px;
488     padding-bottom: max(10px, env(safe-area-inset-bottom));
489   }
490
```

```
491   .message-composer textarea {
492     min-height: 48px;
493     max-height: 120px;
494   }
495
496   .message-composer-footer button {
497     min-width: 70px;
498   }
499 }
500
501 @media (max-width: 380px) {
502   .message-list {
503     padding: 10px;
504   }
505
506   .message-row {
507     max-width: 94%;
508   }
509
510   .message-composer {
511     padding-left: 8px;
512     padding-right: 8px;
513   }
514
515   .message-composer-footer {
516     gap: 6px;
517   }
518
519   .message-composer-footer span {
520     font-size: 11px;
521   }
522
523   .message-composer-footer button {
524     padding-left: 14px;
525     padding-right: 14px;
526   }
527 }
528
529 .conversation-item:focus-visible {
530   outline-offset: -3px;
531 }
532
533 .conversation-item-active {
534   box-shadow: inset 3px 0 var(--color-primary);
535 }
536
537 .message-bubble span {
538   font-size: 12px;
539 }
540
541 .chat-recovery {
542   padding: 8px 12px;
543   flex-shrink: 0;
544 }
545
546 @media (max-width: 800px) {
547   .message-composer textarea { font-size: 16px; resize: none; }
548 }
```

src/styles/chat.module.css

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/styles/chat.module.css
2  +++ after/src/styles/chat.module.css
3  @@ -1,7 +1,7 @@
4      .messages-page {
5          width: 100%;
6          max-width: 1200px;
7  -     height: calc(100dvh - var(--navbar-height));
8  +     height: var(--chat-available-height, calc(100dvh - var(--navbar-height)));
9          margin: 0 auto;
10         padding: 24px 20px;
11         box-sizing: border-box;
12  @@ -396,7 +396,7 @@
13
14     @media (max-width: 800px) {
15         .messages-page {
16  -         height: calc(100dvh - var(--navbar-height));
17  +         height: var(--chat-available-height, calc(100dvh - var(--navbar-height)));
18         padding: 10px;
19     }
20
21  @@ -433,7 +433,7 @@
22
23     @media (max-width: 600px) {
24         .messages-page {
25  -         height: calc(100dvh - var(--navbar-height));
26  +         height: var(--chat-available-height, calc(100dvh - var(--navbar-height)));
27         padding: 0;
28     }
29
30  @@ -542,3 +542,7 @@
31     padding: 8px 12px;
32     flex-shrink: 0;
33 }
34 +
35 +@media (max-width: 800px) {
36 +    .message-composer textarea { font-size: 16px; resize: none; }
37 +}
```

```
1  const notificationLabels = {
2    NEW_MESSAGE: "New message", CHAT_MESSAGE: "New message",
3    FRIEND_REQUEST: "Friend request", FRIEND_REQUEST_ACCEPTED: "Friend request accepted",
4  };
5  export function notificationLabel(type) {
6    return notificationLabels[type] || "Activity";
7  }
8
9  /** Offset-free backend LocalDateTime is a wall clock, not a known UTC instant. */
10 export function formatTimestamp(value, timeOnly = false, locale) {
11   if (typeof value !== "string" || !value.trim()) return "-";
12   const wallClock = /^\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}(?:\\.\\d{2}(?:\\.\\d+)?)?$/i.test(value);
13   const zoned = /^(?:Z|[-+]\\d{2}:\\d{2})$/i.test(value);
14   if (!wallClock && !zoned) return "-";
15   // UTC formatting here preserves the supplied clock fields; it does not infer UTC storage.
16   const date = new Date(wallClock ? `${value}Z` : value);
17   if (Number.isNaN(date.getTime())) return "-";
18   if (wallClock && date.toISOString().slice(0, 16) !== value.slice(0, 16)) return "-";
19   const options = timeOnly ? { hour: "2-digit", minute: "2-digit" }
20     : { year: "numeric", month: "short", day: "numeric", hour: "2-digit", minute: "2-digit" };
21   if (wallClock) options.timeZone = "UTC";
22   return new Intl.DateTimeFormat(locale, options).format(date);
23 }
```

```
1 --- /dev/null
2 +++ after/src/utils/displayText.js
3 @@ -0,0 +1,23 @@
4 +const notificationLabels = {
5 +  NEW_MESSAGE: "New message", CHAT_MESSAGE: "New message",
6 +  FRIEND_REQUEST: "Friend request", FRIEND_REQUEST_ACCEPTED: "Friend request accepted",
7 +};
8 +export function notificationLabel(type) {
9 +  return notificationLabels[type] || "Activity";
10 +}
11 +
12 +/** Offset-free backend LocalDateTime is a wall clock, not a known UTC instant. */
13 +export function formatTimestamp(value, timeOnly = false, locale) {
14 +  if (typeof value !== "string" || !value.trim()) return "-";
15 +  const wallClock = /^(\d{4}-\d{2}-\d{2}T\d{2}:\d{2})(?:\.\d{2})?(?:\.\d+)?$/i.test(value);
16 +  const zoned = /(?:Z|[-+]\d{2}:\d{2})$/i.test(value);
17 +  if (!wallClock && !zoned) return "-";
18 +  // UTC formatting here preserves the supplied clock fields; it does not infer UTC storage.
19 +  const date = new Date(wallClock ? `${value}Z` : value);
20 +  if (Number.isNaN(date.getTime())) return "-";
21 +  if (wallClock && date.toISOString().slice(0, 16) !== value.slice(0, 16)) return "-";
22 +  const options = timeOnly ? { hour: "2-digit", minute: "2-digit" }
23 +    : { year: "numeric", month: "short", day: "numeric", hour: "2-digit", minute: "2-digit" };
24 +  if (wallClock) options.timeZone = "UTC";
25 +  return new Intl.DateTimeFormat(locale, options).format(date);
26 +}
```

tests/displayText.test.js

FULL CURRENT CONTENT

```
1 import test from "node:test";
2 import assert from "node:assert/strict";
3 import { formatTimestamp, notificationLabel } from "../src/utils/displayText.js";
4
5 test("known notification types have readable labels and unknown types stay generic", () => {
6   assert.equal(notificationLabel("NEW_MESSAGE"), "New message");
7   assert.equal(notificationLabel("FRIEND_REQUEST_ACCEPTED"), "Friend request accepted");
8   assert.equal(notificationLabel("PRIVATE_INTERNAL_NAME"), "Activity");
9 });
10 test("invalid or missing timestamps never render Invalid Date", () => {
11   for (const value of [undefined, null, "", "nonsense", "2026-02-31T10:00:00"]) assert.equal(formatTimestamp(value), "-
  > ");
12 });
13 test("offset-free clock fields are preserved across viewer timezones", () => {
14   const previous = process.env.TZ;
15   try {
16     process.env.TZ = "America/New_York";
17     const us = formatTimestamp("2026-09-19T10:15:00", true, "en-GB");
18     process.env.TZ = "Asia/Kolkata";
19     assert.equal(formatTimestamp("2026-09-19T10:15:00", true, "en-GB"), us);
20     assert.equal(us, "10:15");
21     assert.equal(formatTimestamp("2026-09-19T10:15:00Z", true, "en-GB"), "15:45");
22   } finally { if (previous === undefined) delete process.env.TZ; else process.env.TZ = previous; }
23 });
```

tests/displayText.test.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- /dev/null
2 +++ after/tests/displayText.test.js
3 @@ -0,0 +1,23 @@
4 +import test from "node:test";
5 +import assert from "node:assert/strict";
6 +import { formatTimestamp, notificationLabel } from "../src/utils/displayText.js";
7 +
8 +test("known notification types have readable labels and unknown types stay generic",()=>{
9 +  assert.equal(notificationLabel("NEW_MESSAGE"),"New message");
10 +  assert.equal(notificationLabel("FRIEND_REQUEST_ACCEPTED"),"Friend request accepted");
11 +  assert.equal(notificationLabel("PRIVATE_INTERNAL_NAME"),"Activity");
12 +});
13 +test("invalid or missing timestamps never render Invalid Date",()=>{
14 +  for(const value of [undefined,null,"","nonsense","2026-02-31T10:00:00"]) assert.equal(formatTimestamp(value),"
15 +  > -");
16 +});
17 +test("offset-free clock fields are preserved across viewer timezones",()=>{
18 +  const previous=process.env.TZ;
19 +  try {
20 +    process.env.TZ="America/New_York";
21 +    const us=formatTimestamp("2026-09-19T10:15:00",true,"en-GB");
22 +    process.env.TZ="Asia/Kolkata";
23 +    assert.equal(formatTimestamp("2026-09-19T10:15:00",true,"en-GB"),us);
24 +    assert.equal(us,"10:15");
25 +    assert.equal(formatTimestamp("2026-09-19T10:15:00Z",true,"en-GB"),"15:45");
26 +  } finally { if(previous===undefined)delete process.env.TZ;else process.env.TZ=previous; }
27 +});
```

```

1 import { createServer } from "../node_modules/vite/dist/node/index.js";
2 import { fileURLToPath } from "node:url";
3 import assert from "node:assert/strict";
4 import { mkdir } from "node:fs/promises";
5 const { chromium }=await import(process.env.PLAYWRIGHT_MODULE || "playwright");
6 const origin="http://127.0.0.1:41818";
7 process.env.VITE_API_BASE_URL=origin;process.env.VITE_WS_URL=origin.replace("http","ws")+"/ws";
8 const server=await createServer({root:fileURLToPath(new URL("../import.meta.url")),server:{host:"127.0.0.1",port:
> 41818,strictPort:true}});
9 const user={id:1,name:"Review",email:"review@example.invalid"},friend={id:2,name:"Friend"};
10 let browser,oldRoute,failPage=true;
11 const errors=[];
12 try {
13   await server.listen();browser=await chromium.launch({channel:"msedge",headless:true});
14   const page=await browser.newPage({viewport:{width:390,height:900}});
15   page.on("pageerror",e=>errors.push(e.message));
16   await page.addInitScript(user=>{
17     localStorage.setItem("frndbook_access_token","x."+btoa(JSON.stringify({exp:999999999}))+".x");
18     localStorage.setItem("frndbook_refresh_token","fixture");localStorage.setItem("frndbook_user",JSON.stringify(
> user));
19   },user);
20   await page.route("**/*",route=>{
21     const url=new URL(route.request().url());assert.equal(url.origin,origin);
22     if(!url.pathname.startsWith("/api/"))return route.continue();
23     let body=[];
24     if(url.pathname==="/api/users/me")body=user;
25     else if(url.pathname==="/api/users/search") {
26       const query=url.searchParams.get("name"),number=Number(url.searchParams.get("page"));
27       if(query==="old"){oldRoute=route;return;}
28       if(number===1&&failPage)return route.fulfill({status:503,json:{message:"Search temporarily unavailable"}});
29       body={content:[{...friend,name:number===1?"Page two":"Latest result"},number,totalPages:2];
30     } else if(url.pathname.endsWith("unread-count"))body=1;
31     else if(url.pathname==="/api/notifications")body={content:[{id:1,type:"NEW_MESSAGE",message:"Example message"
> ,createdAt:"invalid",read:false},number:0,totalPages:1];
32     else if(url.pathname==="/api/conversations")body=[{id:12,otherUser:friend,updatedAt:"2026-09-19T10:15:00"}];
33     else if(url.pathname.endsWith("/messages"))body={content:[{id:1,conversationId:12,sender:friend,content:"Exam
> ple chat",createdAt:"2026-09-19T10:15:00"}],last:true,totalPages:1];
34     return route.fulfill({json:body});
35   });
36   await page.routeWebSocket("**/*",socket=>socket.onMessage(frame=>{
37     if(/^((CONNECT|STOMP)/.test(String(frame)))socket.send("CONNECTED\nversion:1.2\nheart-beat:0,0\n\n0");
38     if(String(frame).startsWith("DISCONNECT"))socket.close();
39   }));
40   await page.goto(origin+"/");assert.equal(await page.title(),"FrndBook");
41   const search=page.getByRole("textbox",{name:"Search people by name"});
42   await search.fill("old");await page.getByRole("status").filter({hasText:"Searching..."}).waitFor();
43   assert.equal(await page.getByText("No users found.",{exact:true}).count(),0);
44   for(let i=0;!oldRoute&&i<100;i++)await new Promise(r=>setTimeout(r,10));assert.ok(oldRoute);
45   await search.fill("new");await page.getByText("Latest result",{exact:true}).waitFor();
46   await oldRoute.fulfill({json:{content:[{...friend,name:"Obsolete result"},number:0,totalPages:1]}).catch(()=>{
> });
47   assert.equal(await page.getByText("Obsolete result",{exact:true}).count(),0);
48   await page.getByRole("button",{name:"Next",exact:true}).click();
49   await page.getByRole("button",{name:"Retry search"}).waitFor();
50   failPage=false;await page.getByRole("button",{name:"Retry search"}).click();await page.getByText("Page two",{ex
> act:true}).waitFor();
51   await search.fill("");assert.equal(await page.getByText("Page two",{exact:true}).count(),0);
52   assert.equal(await page.getByText("Searching...",{exact:true}).count(),0);
53   await page.goto(origin+"/notifications");await page.getByText("New message",{exact:true}).waitFor();
54   assert.equal(await page.getByText("NEW_MESSAGE",{exact:true}).count(),0);
55   assert.equal(await page.getByText("Invalid Date",{exact:true}).count(),0);
56   await page.goto(origin+"/messages");await page.locator(".conversation-item").first().click();
57   await page.getByRole("textbox",{name:"Message",exact:true}).waitFor();
58   // Simulate an overlay keyboard: visual viewport shrinks while layout viewport stays tall.
59   await page.evaluate(()=>{
60     Object.defineProperty(window.visualViewport,"height",{configurable:true,get:()=>400});
61     window.visualViewport.dispatchEvent(new Event("resize"));
62   });
63   await page.waitForFunction(()=>{
64     const composer=document.querySelector(".message-composer");

```

```
65     return composer && composer.getBoundingClientRect().bottom<=401;
66   });
67   const bounds=await page.locator(".message-composer").boundingBox();assert.ok(bounds.y>=0&&bounds.y+bounds.height
>   t<=401);
68   assert.equal(await page.getByRole("textbox",{name:"Message",exact:true}).evaluate(el=>getComputedStyle(el).font
>   Size),"16px");
69   if(process.env.RELEASE_SCREENSHOT_DIR){await mkdir(process.env.RELEASE_SCREENSHOT_DIR,{recursive:true});await p
>   age.screenshot({path:process.env.RELEASE_SCREENSHOT_DIR+"/mobile-keyboard-layout.png",fullPage:true});}
70   await page.evaluate(()=>{delete window.visualViewport.height;window.visualViewport.dispatchEvent(new Event("res
>   ize"));});
71   await page.setViewportSize({width:1280,height:900});
72   await page.waitForFunction(()=>!document.querySelector(".messages-page").style.getPropertyValue("--chat-availab
>   le-height"));
73   await page.getByRole("textbox",{name:"Message",exact:true}).waitFor();
74   assert.deepEqual(errors,[]);
75   console.log("PASS: title, pending search and stale-response isolation, failed-page retry, empty query cleanup,
>   readable notifications, invalid dates, simulated keyboard viewport and desktop restoration.");
76 } finally {await browser?.close();await server.close();}
```

```

1  --- /dev/null
2  +++ after/tests/release.browser.mjs
3  @@ -0,0 +1,76 @@
4  +import { createServer } from "../node_modules/vite/dist/node/index.js";
5  +import { fileURLToPath } from "node:url";
6  +import assert from "node:assert/strict";
7  +import { mkdir } from "node:fs/promises";
8  +const { chromium }=await import(process.env.PLAYWRIGHT_MODULE || "playwright");
9  +const origin="http://127.0.0.1:41818";
10 +process.env.VITE_API_BASE_URL=origin;process.env.VITE_WS_URL=origin.replace("http","ws")+"/ws";
11 +const server=await createServer({root:fileURLToPath(new URL("..",import.meta.url)),server:{host:"127.0.0.1",port
  > :41818,strictPort:true}});
12 +const user={id:1,name:"Review",email:"review@example.invalid"},friend={id:2,name:"Friend"};
13 +let browser,oldRoute,failPage=true;
14 +const errors=[];
15 +try {
16 +   await server.listen();browser=await chromium.launch({channel:"msedge",headless:true});
17 +   const page=await browser.newPage({viewport:{width:390,height:900}});
18 +   page.on("pageerror",e=>errors.push(e.message));
19 +   await page.addInitScript(user=>{
20 +     localStorage.setItem("frndbook_access_token","x."+btoa(JSON.stringify({exp:9999999999}))+".x");
21 +     localStorage.setItem("frndbook_refresh_token","fixture");localStorage.setItem("frndbook_user",JSON.stringify
  > (user));
22 +   },user);
23 +   await page.route("**/*",route=>{
24 +     const url=new URL(route.request().url());assert.equal(url.origin,origin);
25 +     if(!url.pathname.startsWith("/api/"))return route.continue();
26 +     let body=[];
27 +     if(url.pathname==="/api/users/me")body=user;
28 +     else if(url.pathname==="/api/users/search") {
29 +       const query=url.searchParams.get("name"),number=Number(url.searchParams.get("page"));
30 +       if(query==="old"){oldRoute=route;return;}
31 +       if(number===1&&failPage)return route.fulfill({status:503,json:{message:"Search temporarily unavailable"}})
  > ;
32 +       body={content:[{...friend,name:number===1?"Page two":"Latest result"},number,totalPages:2];
33 +     } else if(url.pathname.endsWith("unread-count"))body=1;
34 +     else if(url.pathname==="/api/notifications")body={content:[{id:1,type:"NEW_MESSAGE",message:"Example message
  > ",createdAt:"invalid",read:false}],number:0,totalPages:1};
35 +     else if(url.pathname==="/api/conversations")body=[{id:12,otherUser:friend,updatedAt:"2026-09-19T10:15:00"}];
36 +     else if(url.pathname.endsWith("/messages"))body={content:[{id:1,conversationId:12,sender:friend,content:"Exa
  > mple chat",createdAt:"2026-09-19T10:15:00"}],last:true,totalPages:1};
37 +     return route.fulfill({json:body});
38 +   });
39 +   await page.routeWebSocket("**/*",socket=>socket.onMessage(frame=>{
40 +     if(/^([CONNECT|STOMP])/ .test(String(frame)))socket.send("CONNECTED\nversion:1.2\nheart-beat:0,0\n\n\n");
41 +     if(String(frame).startsWith("DISCONNECT"))socket.close();
42 +   }));
43 +   await page.goto(origin+"/");assert.equal(await page.title(),"FrndBook");
44 +   const search=page.getByRole("textbox",{name:"Search people by name"});
45 +   await search.fill("old");await page.getByRole("status").filter({hasText:"Searching..."}).waitFor();
46 +   assert.equal(await page.getByText("No users found.",{exact:true}).count(),0);
47 +   for(let i=0;!oldRoute&&i<100;i++)await new Promise(r=>setTimeout(r,10));assert.ok(oldRoute);
48 +   await search.fill("new");await page.getByText("Latest result",{exact:true}).waitFor();
49 +   await oldRoute.fulfill({json:{content:[{...friend,name:"Obsolete result"}],number:0,totalPages:1}}).catch(()=>
  > {});
50 +   assert.equal(await page.getByText("Obsolete result",{exact:true}).count(),0);
51 +   await page.getByRole("button",{name:"Next",exact:true}).click();
52 +   await page.getByRole("button",{name:"Retry search"}).waitFor();
53 +   failPage=false;await page.getByRole("button",{name:"Retry search"}).click();await page.getByText("Page two",{e
  > xact:true}).waitFor();
54 +   await search.fill("");assert.equal(await page.getByText("Page two",{exact:true}).count(),0);
55 +   assert.equal(await page.getByText("Searching...",{exact:true}).count(),0);
56 +   await page.goto(origin+"/notifications");await page.getByText("New message",{exact:true}).waitFor();
57 +   assert.equal(await page.getByText("NEW_MESSAGE",{exact:true}).count(),0);
58 +   assert.equal(await page.getByText("Invalid Date",{exact:true}).count(),0);
59 +   await page.goto(origin+"/messages");await page.locator(".conversation-item").first().click();
60 +   await page.getByRole("textbox",{name:"Message",exact:true}).waitFor();
61 +   // Simulate an overlay keyboard: visual viewport shrinks while layout viewport stays tall.
62 +   await page.evaluate(()=>{
63 +     Object.defineProperty(window.visualViewport,"height",{configurable:true,get:()=>400});

```

```
64 +   window.visualViewport.dispatchEvent(new Event("resize"));
65 + });
66 + await page.waitForFunction(()=>{
67 +   const composer=document.querySelector(".message-composer");
68 +   return composer && composer.getBoundingClientRect().bottom<=401;
69 + });
70 + const bounds=await page.locator(".message-composer").boundingBox();assert.ok(bounds.y>=0&&bounds.y+bounds.height<=401);
71 + assert.equal(await page.getByRole("textbox",{name:"Message",exact:true}).evaluate(el=>getComputedStyle(el).fontSize),"16px");
72 + if(process.env.RELEASE_SCREENSHOT_DIR){await mkdir(process.env.RELEASE_SCREENSHOT_DIR,{recursive:true});await page.screenshot({path:process.env.RELEASE_SCREENSHOT_DIR+"/mobile-keyboard-layout.png",fullPage:true});}
73 + await page.evaluate(()=>{delete window.visualViewport.height>window.visualViewport.dispatchEvent(new Event("resize"))});
74 + await page.setViewportSize({width:1280,height:900});
75 + await page.waitForFunction(()=>!document.querySelector(".messages-page").style.getPropertyValue("--chat-available-height"));
76 + await page.getByRole("textbox",{name:"Message",exact:true}).waitFor();
77 + assert.deepEqual(errors,[]);
78 + console.log("PASS: title, pending search and stale-response isolation, failed-page retry, empty query cleanup, readable notifications, invalid dates, simulated keyboard viewport and desktop restoration.");
79 +} finally {await browser?.close();await server.close();}
```